Communications Engineering

https://doi.org/10.1038/s44172-026-00691-8

Article in Press

A deep learning framework for scientific chart data extraction and reconstruction

Received: 23 September 2025

Accepted: 7 May 2026

Cite this article as: Yuan, Y., Liang, S., Zhang, J. *et al.* A deep learning framework for scientific chart data extraction and reconstruction. *Commun Eng* (2026). https://doi.org/10.1038/s44172-026-00691-8

Yang Yuan, Sihan Liang, Ji Zhang, Yangang Wang & Zongguo Wang

A Deep Learning Framework for Scientific Chart Data Extraction and Reconstruction

Yang Yuan[1,2], Sihan Liang[1,2], Ji Zhang[1,2], Yangang Wang[1,2], and Zongguo Wang[1,2,*]

[1]Computer Network Information Center, Chinese Academy of Sciences, Beijing 100083, China
[2]University of Chinese Academy of Sciences, Beijing 100190, China
[*]wangzg@cnic.cn

ABSTRACT

Scientific figures encapsulate key empirical evidence and are critical to data-driven discovery and scientific integrity assessment. Despite substantial advances in scientific natural language processing for scientific document analysis, automatic quantitative extraction of quantitative data from visual plots remains a challenge due to graphical heterogeneity, noise, and non-standardized formats. In this paper, we present ChartRecover, a deep learning-based framework for end-to-end extraction and reconstruction of chart data from scientific figures. ChartRecover adopts object detection to robustly identify graphical elements across diverse plot types, and introduces a coordinate transformation strategy that aligns visual pixel coordinates with real-world numerical values through axis tick-mark alignment and adaptive conversion. We benchmark ChartRecover across a wide range of figure styles and perturbation conditions, demonstrating strong generalization and high-fidelity recovery performance. The extracted structured data enables downstream applications such as structure–property relationship mining and automated scientific verification. Our work advances machine understanding of scientific figures and provide a scalable tool to enhance research transparency, reproducibility, and data accessibility.

Introduction

As a vital medium for disseminating scientific research findings, charts play a crucial role in scientific research papers and technical reports. Chart data not only serves as the basis for supporting scientific conclusions but has also become a cornerstone for maintaining academic research integrity[1]. In fields such as materials science, chemistry, and biomedicine, obtaining chart data from scientific literature has become an important method for accumulating high-quality data under the current new research paradigm[2,3]. However, chart data in scientific literature is usually presented as unstructured images. These scientific images often feature multi-layer coordinate axes, overlapping regions, and diverse plotting standards, which pose substantial challenges to the systematic analysis and reuse of the underlying data.

Therefore, developing efficient and accurate chart data extraction techniques is of great theoretical and practical significance for enabling in-depth mining of scientific data and verifying and reproducing scientific hypotheses[4].

Traditional chart data extraction techniques primarily rely on template matching methods, achieving data extraction by constructing a chart feature template library. Mishchenko et al.[5] proposed a model-based approach for chart data extraction. This method constructs an abstract model of graphical components and integrates techniques (e.g., 3D color histograms) to extract data from various chart types, including bar charts, line charts, pie charts, scatter plots, and area charts. Savva et al. developed the ReVision system[6], which further combines image and text features to classify charts with the assistance of manually specified text component regions. Al-Zaidy et al.[7] proposed a method based on connected component labeling and OCR technology to extract data from simple rule bar charts. A notable work in related fields is the synthetic data generation framework proposed by Albuquerque et al.[8], which generates high-dimensional datasets through statistical modeling and

interactive visual specifications. While this approach can leverage existing visualization settings, such as scatter chart matrices, to mimic data trends or reconstruct structures from real-world examples, it is primarily designed as an interactive tool for algorithm simulation and testing. In contrast, chart data extraction or recovery methods focuses on the fully automated extraction of quantitative raw data from a broad spectrum of heterogeneous and non-standardized scientific chart images found in the literature, operating independently of interactive design or specific visualization templates. However, these methods generally rely on pre-designed templates and rules, and manual intervention can introduce additional errors. Similarly, mainstream data extraction tools such as Origin and WebPlotDigitizer rely on manual interaction, cannot handle batch processing, and are difficult to adapt to large-scale data extraction scenarios. These earlier methods typically rely on pre-designed templates, hand-crafted rules, or manual interaction. While effective in constrained settings, they often suffer from limited flexibility and poor generalization. Their performance degrades substantial when faced with complex layouts (e.g., irregular or non-standard axes), diverse plotting conventions across journals, or stylistic variations in real-world figures. Moreover, many such methods lack the ability to deeply parse chart semantics, for instance, automatically distinguish between stacked and non-stacked bar charts, which further restricts their applicability.

In recent years, deep learning has enabled adaptation to complex chart styles, effective analysis of diverse information within charts, and precise identification of key data elements, thanks to its powerful feature learning capabilities. Currently, in scientific literature, commonly used chart data can be broadly categorized into two main types: category-based charts represented by bar charts, and numerical correlation-based charts represented by scatter plots. In category-based charts, Dai et al. proposed Chart decoder[9], which uses GoogLeNet[10] and SVM[11] in conjunction with traditional preprocessing methods to achieve simple bar chart data extraction. Liu et al.[12] proposed a framework for the automatic data extraction of bar charts and pie charts. This framework builds a VGG16-based classification model[13] and integrates RN[14] and CRNN[15] technologies into the Faster-RCNN architecture[16]. Carderas et al.[17] employed a traditional-deep learning hybrid method combining Inception v3[18] and SRCNN[19] to develop a method focused on automatic bar chart data extraction.

In numerical correlation-based charts, Poco et al.[20] combined Darknet[21] and Tesseract[22] to achieve automated recovery of visual encoding but were unable to extract data values. Shtok et al. developed the CHARTER framework[23] based on Faster-RCNN[16] and CenterNet[24], employing a four-stage process to handle pie charts and scatter charts. However, the chart element detection stage relies solely on synthetic data for training. This leads to a noticeable deviation from real-world application scenarios. Ma et al.[25] proposed dedicated detectors for box-type and point-type data based on Cascade R-CNN[26] and FCN[27], enabling coordinate extraction, and achieved legend matching by constructing a feature matching network. Most information extraction in numerical correlation-based charts focuses on pixel chart extraction. To obtain real data, further implementation of coordinate conversion between pixel coordinates and real numerical values is required.

Cliche et al. developed Scatteract[28], which concentrates on scatter chart extraction. Due to the axes' tick positioning issues, it cannot guarantee data extraction accuracy. Luo et al. developed the ChartOCR framework[29], which enables data extraction from charts such as bar charts and line charts. It relies solely on the tick value text information for coordinate conversion and axis mapping relationships through simple linear fitting of the maximum and minimum values, resulting in substantial deviations in practical applications. Lal et al. constructed LineFormer[30], which focuses on line chart understanding through instance segmentation and extracts pixel-level coordinates of curve elements. Shivasankaran et al. constructed LineEX[31], which focuses on line charts and achieves true coordinate extraction for line charts. In coordinate conversion, the x-axis and y-axis rely on median fitting, which is easily affected by data shifts and is not applicable in cases of non-linear scales. Kanroo et al. proposed the C3E framework[32]. This framework concentrates on foundational component-level tasks, including chart classification, text detection and recognition, and text role classification. However, it does not attempt to restore real data coordinates. Additionally, Yan et al. proposed the CACHED[33] framework, which innovatively introduces a context-aware mechanism to substantial improve detection accuracy, focusing on chart element recognition.

Previous studies have demonstrated that chart data extraction from scientific literature typically consists of two key steps:

(1)Pixel coordinate extraction: This step involves identifying key chart elements (e.g., coordinate axes, tick values, legends, and data points) and acquiring their pixel coordinates. (2)Coordinate conversion: This step converts the extracted pixel coordinates into actual scientific data values. However, most current research remains at the pixel coordinate extraction stage and has not yet systematically addressed the core challenge of true coordinate recovery. In fact, only obtaining high-precision real data coordinates can meet the basic requirements of data-driven analysis and scientific discovery[34]. Therefore, under the current research paradigm shift toward automation and high-quality data accumulation[35,36], it is imperative to focus on the crucial process of extracting real values from chart data and developing automated processing tools with high accuracy and universality.

This research is centered around two of the most representative types of scientific figures, bar charts and scatter charts, with the aim of constructing an end-to-end automated extraction framework named ChartRecover for chart recognition and data coordinate recovery. The framework employs deep learning as its core technological foundation, specifically addressing the following three key challenges: (1) accurately detecting key elements and data point locations within charts to provide pixel-level high-quality input for subsequent coordinate mapping; (2) designing a high-precision pixel-coordinate mapping algorithm to overcome core difficulties in converting pixel information into numerical values, enabling accurate recovery of true data values; (3) constructing a unified deep learning architecture that integrates element recognition and coordinate transformation, achieving efficient extraction, precise parsing, and structured representation of data from scientific figures.

Result

Chart data extraction framework ChartRecover

This research addresses the challenges of complex information sources and diverse layouts in charts. By employing a multi-model collaborative approach, we have designed an end-to-end automated chart data extraction framework. The overall architecture, as illustrated in Figure 1, comprises four primary modules: chart input, element detection, coordinate conversion, and data output. Among these, the element detection and coordinate conversion modules form the core of the framework. The element detection module takes chart images as input and outputs the pixel coordinates of various elements within the image. The coordinate conversion module uses the tick information and pixel coordinates of the data output by the element detection module as input, converting them into meaningful real data coordinates, and finally outputs normalized numerical results.

The element detection module includes chart element detection and data element detection. This module is based on object detection methods. Chart element detection is used to accurately identify common components in charts, such as axes, tick marks, and legends, providing support for chart structure analysis and coordinate system construction. Data element detection is trained separately for scatter charts and bar charts, focusing on identifying geometric features such as the center position of data points, the vertices, bottom edge, and width of the bars.

In the coordinate conversion module, we use a tick value-mark coordinate alignment algorithm to precisely match tick values with the pixel positions of their corresponding tick marks, substantial improving the fitting accuracy of the coordinate axis mapping function. This enables intelligent detection and adaptive modeling of coordinate axis types. Based on this, the framework can automatically perform precise restoration of data from pixel coordinates to real values for both scatter charts and bar charts. Additionally, to address the practical issue of diverse bar chart styles, the framework incorporates strategies such as direction determination (horizontal or vertical), baseline identification, and stacked structure analysis, supporting data extraction from bar charts of various forms, thereby further enhancing the system's versatility and practicality.

Evaluation results of pixel coordinates extraction

In the evaluation of pixel coordinate extraction results, we selected CACHED[33], a cutting-edge model in the field of bar chart detection and chart element recognition, as the baseline model. We also selected two state-of-the-art (SOTA) models in the field of object detection, DINO[37] and DDQ[38], as comparison benchmarks. The element detection module in ChartRecover is implemented based on the Co-DINO[39] model, and all experiments were conducted under the framework of MMDetection[40]

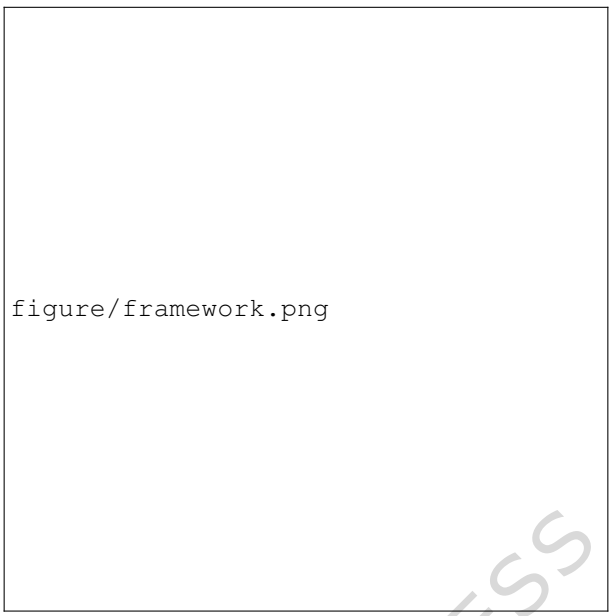


Figure 1. Chart Data Extraction Framework Process. It comprises two core modules: the Element Detection Module (detects and acquires pixel coordinates for elements such as axes, tick, and data points) and the Coordinate Conversion Module (enables precise mapping from pixels to real values).

based on PyTorch. For model evaluation, we adopted the Average Precision (AP) series metrics commonly used in the object detection field. AP is the average precision calculated at multiple intersection over union (IoU) thresholds, specifically sampled from 0.5 to 0.95 in steps of 0.05, comprehensively reflecting the model's detection performance at different levels of strictness. AP_{50} and AP_{75} represent the precision at IoU thresholds of 0.5 and 0.75, respectively. The former measures detection capability under loose matching conditions, while the latter evaluates localization precision under strict matching conditions; AP_S, AP_M, and AP_L are designed to evaluate object detection performance for targets of different sizes, specifically small (area $< 32^2$ pixels), medium ($32^2 \leq$ area $\leq 96^2$ pixels), and large (area $> 96^2$ pixels) objects, comprehensively reflecting the model's ability to recognize chart elements of varying sizes.

Extraction of pixel coordinates of chart elements

We evaluated pixel coordinate extraction results based on the PMC dataset[41]. Experimental results demonstrate that the Co-DINO model adopted in our framework effectively enhances chart detection accuracy through its collaborative optimization strategy, granting it superior performance in analyzing complex chart structures. As shown in Table 1, our model demonstrates comprehensive and balanced performance advantages in pixel coordinate extraction, achieving AP improvements of 3.7%, 2.4%, and 10.2% over DINO, DDQ, and CACHED, respectively. Regarding localization accuracy, it leads in both key metrics AP_{50} and AP_{75}, fully reflecting its precise bounding box regression capability. Notably, our model excels in small object detection ($AP_S = 0.729$). It also maintains strong performance for medium-sized objects ($AP_M = 0.858$) and large objects ($AP_L = 0.902$). This full-scale excellence enables it to effectively handle diverse complex chart analysis tasks, demonstrating greater practical value in real-world applications.

The impact of ticks on the evaluation of pixel coordinate extraction of chart elements

As the core bridge connecting coordinate axes to data element detection points, the accurate identification of tick information (including tick values and tick marks) is crucial for converting pixel coordinates to real-world coordinates. The precise localization of tick marks establishes the physical reference foundation of the coordinate axis system, serving as a

Table 1. Chart element detection evaluation results. Bold values indicate the best performance for each metric.

Model	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
CACHED	0.729	0.845	0.790	0.602	0.763	0.939
DINO	0.774	0.898	0.819	0.694	**0.899**	0.855
DDQ	0.784	0.912	0.834	0.710	0.841	0.848
Ours	**0.803**	**0.915**	**0.847**	**0.729**	0.858	**0.902**

prerequisite for subsequent coordinate mapping (see Methods section). However, due to their typically tiny size and simple linear or point-like structures, tick marks are often obscured by neighboring elements such as axis lines, labels, or chart curves. This results in a distinct tiny object detection challenge, where bounding boxes for tick marks are often only 10×10 pixels—substantial smaller than other chart elements.

From the classification evaluation of chart elements (see Supplementary Table 6), it is evident that tick marks as core components of the X/Y axis area—exhibit average precision (AP) values consistently lower than those of other categories. This has become one of the primary bottlenecks constraining overall pixel extraction accuracy. Detailed evaluation data (see Supplementary Table 1) reveals that Co-DINO demonstrates a distinct advantage over other models in the tick mark detection task. Under the stricter AP_{75} metric, Co-DINO achieves over a 90% improvement compared to the baseline model CACHED, consistent with the overall evaluation results presented earlier (Table 1).

Data element pixel coordinate extraction

To further evaluate the performance of our framework in pixel coordinate extraction for bar charts and scatter plots, tests were conducted separately for the two types of chart detection tasks. In the scatter chart detection task, the model accurately captured the spatial relationships between data points under complex distribution conditions through its localization capabilities. Given that the pixel bounding boxes for scatter points are only 7×7 pixels (totaling 49 pixels), this task is a typical small object detection problem. At an IoU threshold of 0.5, the predicted box must overlap with the ground truth box by nearly 32 pixels (65.3%); when the IoU threshold is raised to 0.75, nearly 42 pixels (85.7%) of overlap is required. As shown in Table 2, under these stringent localization conditions, the Co-DINO model achieves an AP of 0.535, with an AP_{50} of 0.955. Its AP_{75} value of 0.718 substantial outperforms both the DINO and DDQ models.

Table 2. Scatter detection evaluation results. Bold values indicate the best performance for each metric.

Model	AP	AP_{50}	AP_{75}	AP_S
DINO	0.448	0.920	0.505	0.513
DDQ	0.424	0.825	0.407	0.424
Ours	**0.535**	**0.955**	**0.718**	**0.602**

In the bar chart detection task, our framework also demonstrates substantial advantages. Table 3 presents the bar detection evaluation results for each model. As shown in Table 3, the Co-DINO model achieves the best performance with an AP value of 0.707. Its outstanding AP_{75} and AP_L scores indicate its superiority in locating the endpoints of column-shaped objects. Notably, the model maintains leadership in both medium-scale (AP_M) and small-scale (AP_S) targets, demonstrating strong adaptability to column width-height ratios. Additionally, we separately evaluated horizontal and vertical columns within the bar chart, with results presented in Supplementary Tables 2 and 3.

Table 3. Bar detection evaluation results. Bold values indicate the best performance for each metric.

Model	AP	AP_{50}	AP_{75}	AP_S	AP_M	AP_L
CACHED	0.692	0.858	0.773	0.514	0.856	0.972
DINO	0.684	0.858	0.778	0.489	0.873	0.967
DDQ	0.690	0.856	0.779	0.496	0.869	0.965
Ours	**0.707**	**0.868**	**0.790**	**0.514**	**0.888**	**0.973**

Evaluation of true value coordinate extraction results

From Figure 1, we can observe that coordinate extraction errors in chart data points primarily stem from two parts: first, the accumulation and transfer of pixel positioning errors; second, mapping errors during the conversion process between pixel coordinates and true coordinates. Unlike traditional methods such as ChartOCR and LineEX that rely solely on tick value-based coordinate conversion, our framework introduces a coordinate conversion mechanism and mapping based on the collaborative alignment of tick marks and tick values, precisely matching tick values with tick marks (see Methods section for details). These improved methods reduce cumulative errors during coordinate extraction by accurately capturing chart tick features and performing effective modeling.

Figure 2 displays the coordinate extraction results of numerical points in scientific literature by ChartRecover, visually presenting both the framework-extracted values and the actual scientific data values. As shown in Figure 2, within the permissible error range, the data point extraction performed by our framework fundamentally satisfies the requirements for scientific data analysis and restoration.

Table 4. Coordinate conversion evaluation results on the UB PMC 2022 and CHART-Info 2024 data set. Bold values indicate the best performance for each metric.

Dataset	Metric6a		Metric6b	
	UB PMC 2022	CHART-Info 2024	UB PMC 2022	CHART-Info 2024
Scatter	0.9360	0.9328	0.8674	0.8975
Horizontal bar	0.8978	0.8489	0.7924	0.8095
Vertical bar	0.8942	0.9129	0.8568	0.8705

To comprehensively evaluate the ChartRecover framework's performance in extracting true coordinates of data points, we conducted systematic experiments using the evaluation metrics Metric6a and Metric6b proposed by CHART-Info[42] as performance benchmarks. These metrics focus on the localization accuracy of pixel-level bounding boxes and the extraction accuracy of true coordinate values, respectively. Our model is tested on both UB PMC 2022 and the recent CHART-Info 2024[43] data sets, and the results are shown in Table 4, it demonstrates that our framework exhibits outstanding performance in both pixel-level localization and coordinate extraction. For pixel-level localization (Metric6a), Metric6a of the scatter chart achieved a high score of 0.936, and that of the bar chart exceed 0.89 on UB PMC 2022 data set. While Metric6a of the scatter chart achieved a high score of 0.933, and the minima of bar chart also reached 0.849 on CHART-Info 2024 data set. For the more challenging metric of true coordinate extraction (Metric6b), it still maintains a high level (Metric6b of scatter chart and bar chart reached nearly or above 0.8). These results highlight the model's robust capability in precisely capturing chart visual elements and true-value recovery.

From Table 4, although ChartRecover was not trained on the CHART-Info 2024 training set, our method still achieves

excellent performance across Metric6a and Metric6b, fully confirming its remarkable generalization capability. To the best of our knowledge, there is currently no evaluation on Metric6a and Metric6b for the CHART-Info 2024 benchmark, and we hope our method can serve as a reference baseline for subsequent research on this benchmark. In addition, we further compared the performance of typical methods with our method on UB PMC 2022 data set, ChartOCR and the top-ranked team of the ICPR 2022 Chart-Infographics Challenge. The comparisons show that ChartRecover outperforms both methods, which further proves the advantage of ChartRecover on real data coordinate recovery. Detailed comparative results and discussions are presented in Supplementary Tables 4 and 5, respectively.

Error threshold evaluation and ablation experiments

Since errors inherently exist objectively, scientific research often establishes reasonable tolerance ranges for errors under specific research objectives. Therefore, we further conducted experiments and analyzed results by introducing an error threshold to evaluate the performance enhancement of the innovative coordinate transformation algorithms (tick value-mark alignment algorithm and mapping function algorithms) included in ChartRecover. Specific findings are presented in Table 5, it demonstrates that when the relative error in ChartRecover-extracted data is 10%, the data point recognition accuracy exceeds 80% for all chart types. Even under the strict 2% error condition, the bar chart maintains a data point recognition accuracy above 70%. Compared to results without alignment algorithms, this method exhibits superior performance across all tested conditions. Table 5 also gives an error threshold evaluation on the CHART-Info 2024 benchmark using the model which trained on UB PMC 2022. As an updated benchmark, CHART-Info 2024 contains a larger number of images and more diverse chart types, making it more representative of real-world scientific chart parsing scenarios. We can see that vertical bar charts exhibit the best overall performance, while the metrics of scatter and horizontal bar charts steadily improve as the error threshold increases and consistently maintain a high performance level. Moreover, compared with the UB PMC 2022 benchmark, several evaluation metrics are even further improved. These results demonstrate the strong adaptability of ChartRecover to complex and large-scale datasets and verify its robust generalization capability. Even when facing diverse chart distributions from a new benchmark, ChartRecover can stably produce high-precision coordinate recovery results, fully confirming its outstanding performance and reliability in real data coordinate extraction and restoration tasks.

Table 5. Error threshold evaluation results (Error threshold* indicates that the alignment algorithm is not used). Bold values highlight the performance improvement achieved by the alignment algorithm on the UB PMC 2022 dataset.

Dataset	Error threshold(2022)*			Error threshold(2022)			Error threshold(2024)		
	2%	5%	10%	2%	5%	10%	2%	5%	10%
Scatter	0.5762	0.7134	0.7778	**0.6388**	**0.7526**	**0.8024**	0.6766	0.7837	0.8258
Horizontal bar	0.6947	0.7927	0.8479	**0.7184**	**0.8089**	**0.8581**	0.7039	0.7882	0.8499
Vertical bar	0.7008	0.8128	0.8655	**0.7691**	**0.8539**	**0.8863**	0.8237	0.8899	0.9143

Methods

Dataset

The UB PMC 2022 dataset is a real-world chart dataset provided by ICPR 2022[41], with all data collected from PubMed Central (PMC). It contains a wide range of high-quality chart samples. This dataset has been manually annotated with precision and is suitable for multiple tasks such as data extraction and chart element detection. The Adobe Synthetic dataset originates from ICDAR 2019[44] and is a synthetic dataset containing samples of different chart types, which was generated using Matplotlib

and has a large number of data samples. For the data element detection task, to compensate for the lack of scatter charts and bar charts in the PMC dataset, we selected supplementary samples from the Adobe dataset to enrich the training data, enhancing its diversity and representativeness. Notably, horizontal and vertical bar charts were treated as the same category during model training. For the chart element detection task, the PMC dataset provided sufficient samples, thus no additional synthetic data was introduced. Table 6 details the distribution of the datasets. The testing phase exclusively utilizes the PMC test dataset, as authentic chart data better reflects the model's performance in real-world applications, ensuring the objectivity of the evaluation. To further verify the model's generalization ability across newer and more diverse benchmarks, we additionally introduce the CHART-Info 2024 test set as an extra testing benchmark. As an updated real-world chart dataset, it features a larger sample size and higher diversity of chart types compared to UB PMC 2022. The strategy of training data fusion enhancement while testing data remains authentic effectively utilizes synthetic data to improve the model's generalization capabilities while ensuring the authenticity and reliability of the final performance evaluation.

Table 6. Distribution of image counts in the dataset

Detection task	Dataset	PMC_train	Adobe_select	PMC 2022_test	CHART-Info 2024_test
Data element	Scatter	541	1200	103	217
	Horizontal bar	1321	1500	113	305
	Vertical bar	496	1500	427	865
Chart element	Element	5612	-	1345	-

Element detection

Our research uses the Co-DINO[39] model as the core architecture to construct an end-to-end framework for detecting chart elements and data information. Based on the Transformer[45] encoder-decoder structure, this model introduces a collaborative hybrid allocation training scheme combined with multiple parallel auxiliary heads for supervised training. This approach preserves the end-to-end advantages of DETR[46] while addressing the sparse supervision issue caused by traditional one-to-one label allocation, substantial improving detection accuracy and generalization capabilities for complex charts. The model architecture is illustrated in Figure 3. The auxiliary head with one-to-many label assignment provides more positive samples, effectively alleviating the problem of insufficient encoder feature learning in traditional DETR caused by too few positive queries in the decoder. For the backbone network, we adopt SwinTransformer-large[47] as the feature extractor. Its robust representation capabilities and hierarchical window attention mechanism are well-suited for handling both tiny elements (e.g., ticks) and macro structures (e.g., title regions) within charts. For auxiliary heads, we utilize ATSS[48] and Faster-RCNN from the Co-DINO framework.

The element detection module aims to accurately identify and localize structural components in charts, including horizontal and vertical axes, axis ticks, tick texts, and chart title areas. With reference to CACHED, we subdivide chart elements into 18 refined categories, as detailed in Supplementary Table 7. This fine-grained classification scheme helps the model better understand the compositional structure of charts.

The data detection component is trained independently for scatter charts and bar charts, focusing on extracting data feature information from the charts: For scatter charts, it primarily detects the center positions of data points, while for bar charts, it concentrates on identifying the vertex coordinates, base positions, and width information of the bars, thereby obtaining high-precision pixel-level data coordinates.

Tick value-mark coordinate alignment algorithm

During the chart element detection phase, the model outputs pixel coordinates for the tick value text areas (x label/y label) and tick mark positions (x tick/y tick). This study first employs the TrOCR-large model to independently recognize tick value text information on each axis, thereby obtaining the semantic information of the tick values. However, in actual charts, affected by factors such as font size variations, line spacing, and layout, the visual center of tick texts often exhibits systematic deviation from the tick marks (as shown in Supplementary Figure 1. If the text center point is directly used as the tick position, it will introduce additional positioning errors and amplify mapping errors in subsequent coordinate value conversions.

To address this issue, we propose a tick value-mark coordinate alignment algorithm. By precisely matching the semantic meaning of tick value text with the pixel coordinates of tick marks, it eliminates the impact caused by tick value deviation, thereby reducing the propagation error of pixel coordinates. The specific process is as follows:

•Coordinate Extraction: Detect and record the visual center point pixel coordinates of tick value texts, as well as the pixel coordinates of tick marks.

•Cost Matrix Construction: Calculate the Euclidean distance between the center point of each tick value text and the corresponding tick mark position.

•Optimal Matching: Apply the Hungarian matching algorithm to the cost matrices for the x-axis and y-axis respectively to establish precise associations between tick values and tick marks.

•Position Reference Correction: Replace the center point positions of tick values with the matched tick mark positions to serve as the geometric reference for coordinate axis mapping.

This method takes tick marks as the physical localization reference, which effectively avoids the impact of text rendering deviation on coordinate accuracy, substantial reduces the propagation error of pixel coordinates, and provides stable and high-precision input for the subsequent fitting of coordinate axis mapping functions.

Axis mapping relationship construction

On the basis of tick value-mark alignment, this study establishes independent mapping relationships between pixel coordinates and physical actual values for the x-axis and y-axis respectively. According to the numerical distribution characteristics of the tick values of each axis, the mapping models are divided into two categories: linear and logarithmic.

(1)Linear coordinate axis:

$$v = kp + b \tag{1}$$

Among the parameters, p represents the pixel coordinate (for the x-axis, only the horizontal component of the pixel coordinate is used; for the y-axis, only the vertical component of the pixel coordinate is used), v represents the real semantic value, k stands for the proportional coefficient(slope), and b is the origin offset(intercept).

The intercept is retained in the mapping function because there exists a scaling relationship and an origin offset between the pixel coordinate system and the real physical coordinate system, causing their positions to be inconsistent. The origin of the pixel coordinate system is located at the top-left corner of the image, while the origin of the real physical coordinate system may be at any arbitrary position within the image. Therefore, using only a linear proportion would introduce a systematic deviation due to the origin mismatch, and the intercept term must be employed to eliminate origin differences.

For linear coordinate axes, we use the linregress function from the SciPy library to calculate k and b based on least squares regression.

(2)Logarithmic coordinate axis:

$$\log_{10} v = kp + b \quad \Rightarrow \quad (v = 10^{kp+b}) \tag{2}$$

The essence of a logarithmic coordinate axis is that $\log_{10}$ and p exhibit a linear relationship. By treating $\log_{10} v$ as a new variable, linear fitting methods can be directly applied to obtain the corresponding k and b values.

Algorithm 1 Chart axis semantic-pixel mapping

Input: Prediction box information for axis labels and ticks, B_{labels}, B_{ticks}, Chart image I

Output: Semantic-pixel mapping relation function, F_{mapping}

$V_{\text{text}} \leftarrow \text{dict}()$
$C_{\text{labels}} \leftarrow \text{list}()$
$C_{\text{ticks}} \leftarrow \text{list}()$
$M_{\text{cost}} \leftarrow \text{emptymatrix}()$
for $i \leftarrow 0$ to $\text{len}(B_{\text{labels}}) - 1$ **do**
    $V_{\text{text}}[i] \leftarrow \text{TrOCR}(I, B_{\text{labels}}[i])$
    $C_{\text{labels}}[i] \leftarrow \text{CalculateCenter}(B_{\text{labels}}[i])$
end for
for $j \leftarrow 0$ to $\text{len}(B_{\text{ticks}}) - 1$ **do**
    $C_{\text{ticks}}[j] \leftarrow \text{CalculateCenter}(B_{\text{ticks}}[j])$
end for
for $i \leftarrow 0$ to $\text{len}(B_{\text{labels}}) - 1$ **do**
    for $j \leftarrow 0$ to $\text{len}(B_{\text{ticks}}) - 1$ **do**
        $M_{\text{cost}}[i][j] \leftarrow \text{EuclideanDistance}(C_{\text{labels}}[i], C_{\text{ticks}}[j])$
    end for
end for
$M_{\text{match}} \leftarrow \text{HungarianMatch}(M_{\text{cost}})$
$D \leftarrow \text{FormPairs}(C_{\text{ticks}}, V_{\text{text}}, M_{\text{match}})$
$F_{\text{line}} \leftarrow \text{LinearRegression}(D, C_{\text{ticks}}, V_{\text{text}})$
$F_{\text{exp}} \leftarrow \text{LogisticRegression}(D, C_{\text{ticks}}, V_{\text{text}})$
$F_{\text{mapping}} \leftarrow \max(R^2(F_{\text{line}}(C_{\text{ticks}}), V_{\text{text}}), R^2(F_{\text{exp}}(C_{\text{ticks}}), V_{\text{text}}))$

The mapping relationship fitting process includes: obtaining the correspondence between pixel coordinates p and semantic values v based on aligned tick points; performing linear regression and log-linear regression fitting for each coordinate axis to obtain corresponding k and b values; calculating the R^2 determination coefficients for both regression models and independently selecting the optimal model for each axis; using the selected model as the mapping function for that axis to convert subsequent data points' pixel coordinates into physical values.

This method substantial enhances the accuracy and stability of coordinate mapping through independent axial modeling and precise tick point locating, while supporting both linear and logarithmic tick scales to accommodate a wider range of scientific graph types. The entire process is detailed in Algorithm 1.

Data point parsing

(1) Scatter Chart Data Parsing

The parsing of scatter charts is relatively straightforward. By utilizing the data information to detect the center pixel coordinates (p_x, p_y) of the model's output scatter points, the true physical coordinates (v_x, v_y) of the scatter points can be calculated through the horizontal and vertical axis mapping functions $v_x = f_x(p_x)$ and $v_y = f_y(p_y)$, respectively.

(2) Bar Chart Data Parsing

Bar charts can be presented in various styles (see Figure 4), and different styles directly influence coordinate conversion methods. Therefore, this study breaks down the bar chart parsing process into four steps: direction determination, baseline positioning, type determination, and numerical calculation.

•Direction Determination: According to the annotation rules of the PMC dataset, the column label axis is defined as the x-axis, and the numerical axis is defined as the y-axis. If the numerical axis is the vertical axis, it is a vertical bar chart; if the numerical axis is the horizontal axis, it is a horizontal bar chart.

•Baseline Positioning: Different column orientations result in different baseline definitions. For vertical bar charts, calculate the coordinates of all bar top and bottom edges, then identify shared boundaries (edges with identical numerical axis coordinates) as the baseline. For horizontal bar charts, calculate the coordinates of all bar left and right edges, then identify shared boundaries as the baseline. This method, based on the statistical characteristics of bar geometric distribution accurately determines the starting point for data measurement across different layouts and drawing styles.

•Type Determination (i.e., Stacked vs. Non-Stacked Cases): Determination is based on the overlapping conditions of column center coordinates in the direction of the numerical axis. If coordinate overlap exists in the direction of the numerical axis, it is classified as a stacked type; if the coordinates are mutually independent in the direction of the numerical axis with no overlap, it is classified as a non-stacked type.

•Numerical Calculation Rules: Under the premise that direction and type are known, calculate the actual value by combining the baseline position. The calculation method is shown in Supplementary Table 8.

In Supplementary Table 8, the end values are obtained by converting the pixel coordinates of the corresponding endpoints of each column through a mapping function. This parsing process demonstrates high robustness and accuracy in bar charts with diverse orientations and complex layouts. The entire process is detailed in Algorithm 2.

Framework performance evaluation metrics

To comprehensively measure the detection performance of scatter charts and bar charts, this study employs a multidimensional evaluation system combining pixel-level and true-value dimensions. The pixel-level evaluation focuses on the localization accuracy of detection models in image space, while the true-value dimension evaluation focuses on the consistency between the semantic values extracted by the model and the true values.

(1) Pixel localization accuracy evaluation Metric6a

This metric originates from CHART-Info and primarily measures the localization error of the detected object within the image's pixel space:

$$\text{score} = \frac{1}{\beta N_{\text{gt}}} \sum_{m,n}^{N_{\text{gt}}} (1 - d_{mn}) \tag{3}$$

For scatter charts, the Euclidean distance is used, and normalization is performed using the shortest side length of the image:

$$d_{ij} = \min\left(1, \frac{\sqrt{(x_i^{\text{pred}} - x_j^{\text{gt}})^2 + (y_i^{\text{pred}} - y_j^{\text{gt}})^2}}{\gamma D}\right) \tag{4}$$

Here, (x, y) represents the pixel coordinates, $D = \min(W, H)$, where W and H denote the width and height of the image respectively. d_{ij} is the distance between each predicted point and the corresponding real spatial pixel point. The scaling factors γ and β are both set to 1 according to CHART-Info. d_{mn} is the matching result obtained by the Hungarian algorithm when computing the cost matrix, and N_{gt} is the number of ground truth samples.

For bar charts, the Manhattan distance is used, and normalization is similarly performed using the shortest side length of the image:

Algorithm 2 Chart data point coordinate conversion

Input: Prediction box information for data points B_{points}, mapping function F_{mapping} (including x-axis mapping F_{mapping}.x and y-axis mapping F_{mapping}.y)

Output: Converted real coordinate values V_{points}

// Scatter Chart data point coordinate conversion
for $k \leftarrow 0$ to $\text{len}(B_{\text{points}}) - 1$ **do**
  $(p_x, p_y) \leftarrow \text{CalculateCenter}(B_{\text{points}}[k])$
  $V_x \leftarrow F_{\text{mapping}}.x(p_x)$
  $V_y \leftarrow F_{\text{mapping}}.y(p_y)$
  $V_{\text{points}}[k] \leftarrow (V_x, V_y)$
end for
// Bar Chart data point coordinate conversion
orientation $\leftarrow \text{DetectBarOrientation}(B_{\text{points}})$
baseline $\leftarrow \text{BarBaselineDetection}(B_{\text{points}}, \text{orientation})$
columntype $\leftarrow \text{ColumnTypeDetection}(B_{\text{points}}, \text{orientation})$
if orientation = horizontal **then**
  for $k \leftarrow 0$ to $\text{len}(B_{\text{points}}) - 1$ **do**
    $(p_{x1}, p_{x2}) \leftarrow (B_{\text{points}}[k].x1, B_{\text{points}}[k].x2)$
    if columntype = stacked **then**
      if $p_{x1} >$ baseline **then**
        $V_{\text{points}}[k] \leftarrow F_{\text{mapping}}.x(p_{x2}) - F_{\text{mapping}}.x(p_{x1})$
      else
        $V_{\text{points}}[k] \leftarrow F_{\text{mapping}}.x(p_{x1}) - F_{\text{mapping}}.x(p_{x2})$
      end if
    else
      if $p_{x1} >$ baseline **then**
        $V_{\text{points}}[k] \leftarrow F_{\text{mapping}}.x(p_{x2})$
      else
        $V_{\text{points}}[k] \leftarrow F_{\text{mapping}}.x(p_{x1})$
      end if
    end if
  end for
end if
if orientation = vertical **then**
  for $k \leftarrow 0$ to $\text{len}(B_{\text{points}}) - 1$ **do**
    $(p_{y1}, p_{y2}) \leftarrow (B_{\text{points}}[k].y1, B_{\text{points}}[k].y2)$
    if columntype = stacked **then**
      if $p_{y1} >$ baseline **then**
        $V_{\text{points}}[k] \leftarrow F_{\text{mapping}}.y(p_{y2}) - F_{\text{mapping}}.y(p_{y1})$
      else
        $V_{\text{points}}[k] \leftarrow F_{\text{mapping}}.y(p_{y1}) - F_{\text{mapping}}.y(p_{y2})$
      end if
    else
      if $p_{y1} >$ baseline **then**
        $V_{\text{points}}[k] \leftarrow F_{\text{mapping}}.y(p_{y1})$
      else
        $V_{\text{points}}[k] \leftarrow F_{\text{mapping}}.y(p_{y2})$
      end if
    end if
  end for
end if

$$d_{ij} = \min\left(1, \frac{\sqrt{(\mathbf{v}_i^{\text{pred}} - \mathbf{v}_j^{\text{gt}})^{\text{T}} V^{-1} (\mathbf{v}_i^{\text{pred}} - \mathbf{v}_j^{\text{gt}})}}{\gamma}\right), \mathbf{v} = [x, y]^{\text{T}} \tag{5}$$

Where $(x1, y1)$ and $(x2, y2)$ represent the upper-left and lower-right corner coordinates of the column pixel box, respectively. $\gamma = 0.05$ and $\beta = 1$ are set according to CHART-Info specifications.

(2) True semantic value accuracy evaluation Metric6b

The metric also originates from CHART-Info, measuring the accuracy of detection results on physical coordinates or numerical labels:

$$score = \frac{1}{N_{\text{gt}}} \sum_{\text{m,n}}^{N_{\text{gt}}} (1 - d_{m,n}) \tag{6}$$

For scatter charts, the distance between predicted and actual values is calculated using Mahalanobis distance:

$$d_{ij} = \min\left(1, \frac{\sqrt{(\mathbf{v}_i^{\text{pred}} - \mathbf{v}_j^{\text{gt}})^{\text{T}} V^{-1} (\mathbf{v}_i^{\text{pred}} - \mathbf{v}_j^{\text{gt}})}}{\gamma}\right), \mathbf{v} = [x, y]^{\text{T}} \tag{7}$$

Where (x, y) represents the x and y coordinates of the scatter points. V^{-1} denotes the inverse of the covariance matrix of the true sample v^{gt}. $\gamma = 1$ follows the CHART-Info configuration.

For bar charts, the distance between predicted and actual values is calculated using Manhattan distance:

$$d_{ij} = min(1, \frac{\left|y_i^{pred} - y_j^{true}\right|}{\gamma\eta}), \eta = min(\sigma, \frac{|\mu|}{20}) \tag{8}$$

Where σ and μ correspond to the standard deviation and mean of the actual sample, respectively. γ is set to 1 according to CHART-Info specifications.

(3) Relative error multi-threshold evaluation

To enhance the interpretability of the metrics in practical applications, this study further refines the error threshold method proposed by FigureSeer[49] and Linear Programming[50], employing relative error as the evaluation metric:

$$\varepsilon_v = \frac{\left|v^{pred} - v^{gt}\right|}{|v^{gt}| + \varepsilon}, v \in (x, y) \tag{9}$$

Where $\varepsilon = 10^{-8}$ is used to prevent division by zero. For scatter charts, v takes the physical coordinate values (x, y); for bar charts, v takes the numerical data.

We calculate accuracy at three thresholds: 2%, 5%, and 10%. First, we perform object matching using the Hungarian algorithm based on the distance metric defined by Metric6a. Next, we count the number of matches where the relative error between the predicted value and the true value falls below the specified error threshold. Finally, we compute the accuracy for each threshold (see Formula 10).

$$\text{Accuracy}(\tau) = \frac{1}{N_{gt}} \sum_{i=1}^{N_{gt}} [\varepsilon_{vi} < \tau] \times 100\%, \tau \in \{0.02, 0.05, 0.1\} \tag{10}$$

(4) Average Precision (AP) evaluation

AP serves as the core evaluation metric in object detection tasks, measuring a model's ability to balance detection accuracy and recall across a range of Intersection over Union (IoU) thresholds.

Use the IoU to judge the matching degree between the predicted box and the ground truth box:

$$\text{IoU} = \frac{\text{area}(B_{pred} \cap B_{gt})}{\text{area}(B_{pred} \cup B_{gt})} \tag{11}$$

Where B_{pred} and B_{gt} represent the coordinates of the predicted bounding box and the ground truth bounding box, respectively.

For each IoU threshold $t \in \{0.5, 0.55, \ldots, 0.95\}$ match according to the following rules: (1) Sort detection boxes in descending order of confidence; (2) If a detection box has IoU $\geq t$ with a ground truth box that has not been matched, mark it as a true positive (TP); otherwise, mark it as a false positive (FP); (3) Each ground truth bounding box can only be matched once.

For each IoU threshold t:

$$\text{Precision}_t = \frac{\text{TP}_t}{N_{\text{pred}}}, \quad \text{Recall}_t = \frac{\text{TP}_t}{N_{\text{gt}}} \tag{12}$$

N_{pred} and N_{gt} represent the total number of predicted boxes and ground truth boxes, respectively.

Apply monotonic decreasing smoothing to the PR curve for each t, then interpolate and compute the average precision at 101 equidistant recall points (0.0 to 1.0, step size 0.01):

$$\text{AP}_t = \frac{1}{101} \sum_{r \in \{0, 0.01, \ldots, 1.0\}} p_{\text{smooth}}(r) \tag{13}$$

The final AP is the average value across all IoU thresholds(0.5 to 0.95):

$$\text{AP} = \frac{1}{10} \sum_{\text{t} \in \{0.5, 0.55, \ldots, 0.95\}} \text{AP}_{\text{t}} \tag{14}$$

The aforementioned AP calculation is applicable to single-category scenarios. For multi-category scenarios, it is only necessary to calculate the AP for each category individually, then sum these AP_{S} and take the average.

AP_{50} and AP_{75} refer to the calculation of AP_{t} with fixed IoU thresholds of 0.5 and 0.75, respectively. AP_{S}, AP_{M}, and AP_{L} are divided based on target sizes. They follow the same calculation method as AP but count the detection results of targets of different scales separately, evaluating the detection performance for small targets (pixel area $< 32^2$), medium targets ($32^2 \leq$ pixel area $\leq 96^2$), and large targets (pixel area $> 96^2$) respectively.

Discussion

The overall error in automated chart data extraction mainly stems from two sources: inaccuracies in pixel-level localization of data elements, and errors introduced during the conversion from pixel coordinates to real-world numerical values. Quantitative evaluation indicates that pixel coordinate extraction for both scatter charts and bar charts is consistently more accurate than the final numerical reconstruction of physical coordinates. This result suggests that the object detection component of ChartRecover achieves stable and reliable performance in the task of element recognition and localization. And the dominant source of error lies in the coordinate conversion stage, especially the systematic errors caused by the fitting of coordinate mapping functions.

A deeper analysis shows that this limitation is largely attributable to the challenge of accurately localizing tick marks, which constitute extremely small visual targets in scientific charts. Although the predicted tick mark positions are close to the ground truth, even minor deviations of a few pixels in their estimated centers can substantially affect the slope and intercept of the fitted coordinate mapping function. These small localization errors therefore translate into systematic deviations in the axis scale mapping. Such errors have a cumulative effect: they directly reduce the accuracy of absolute value reconstruction and

simultaneously amplify relative errors in the subsequent evaluations. While the proposed tick value-mark alignment algorithm can partially mitigate this issue, residual systematic errors remain unavoidable under current resolution constraints.

This error propagation mechanism also explains the seemingly contradictory performance trends observed across different evaluation metrics. As shown in Table 4, scatter charts perform slightly better than bar charts in Metric6b score, but the opposite trend appears in the relative error threshold evaluation shown in Table 5. This discrepancy is primarily driven by data distribution characteristics and the sensitivity of relative error metrics to small numerical values. During the coordinate mapping process, absolute errors that are negligible for large values can result in disproportionately large relative deviations when the true values are close to zero. As shown in Supplementary Figure 2, nearly half of the scatter chart test samples contain at least one coordinate value smaller than 1(gray area means either x or y exceeds 1). Supplementary Figure 3 further confirms that the relative error increases sharply as coordinate approaches zero for both axes. Consequently, scatter charts exhibit weaker performance under relative error-based evaluations, not due to inferior reconstruction capability, but due to the intrinsic amplification effect of the evaluation metric itself.

Beyond coordinate conversion, we also observe that recognition accuracy degrades in several practical scenarios. When tick values are rendered with rotation or italicization (e.g., angled scale text), TrOCR-large may generate incorrect numerical outputs due to distorted character alignment (Supplementary Figure 4(a)). Similarly, when numerical values include non-standard numberical formatting, such as space-separated thousands (e.g., 2 500), the model may misinterpret spaces as decimal points (e.g., 2.500), leading to systematic numerical errors (Supplementary Figure 4(b)). In addition, although the datasets used in this study primarily contain Latin-script numerals, we acknowledge that TrOCR-large is not optimized for multilingual or non-Latin chart text, which may further limit its applicability in diverse scientific figures. It is important to note that OCR is treated as a pluggable module within the ChartRecover framework. The overall coordinate recovery pipeline is not inherently tied to TrOCR-large, and alternative OCR engines with stronger multilingual or chart-aware capabilities can be seamlessly integrated. Future work will explore more robust OCR models and text normalization strategies to improve recognition reliability across diverse scripts, layouts, and formatting styles.

Finally, while ChartRecover framework proposed in this study is currently designed for automatic data extraction from scatter and bar charts, its modular architecture provides a clear pathway for extension to other common scientific chart types. For non-scatter line charts, instance segmentation techniques can be incorporated to capture pixel-level representations of data line segments. For box charts, dedicated detection modules can be designed to identify structural elements such as quartile boundaries and median lines, followed by chart-specific numerical rules to derive the underlying statistics. Histograms extraction can largely reuse the existing bar chart detection workflow, augmented with interval-aware bin association during coordinate transformation. For multi-axis charts, a fine-grained axis classification mechanism can be integrated into the chart element detection stage to distinguish multiple coordinate axes systems (e.g., x1/ x2, y1/ y2), enabling parallel coordinate mapping using the existing algorithm. These extensions fully reuse ChartRecover’ s core modules of element detection and coordinate transformation, allowing efficient scaling to more complex charts without redesigning the overall architecture.

Conclusion

In this work, we address the challenge of automated data extraction and reconstruction from scatter and bar charts in scientific publications by proposing ChartRecover, an end-to-end deep learning framework that covers the entire pipeline from chart element localization to semantic value conversion. Designed for absolute measurement data reconstruction, ChartRecover enables direct restoration of the original numerical values displayed in charts. Its precise coordinate transformation strategy fully preserves the quantitative characteristics of the data, thereby meeting the core requirement for reliable reuse of original experimental data. By integrating fine-grained detection of both chart and data elements with an adaptive tick-mark alignment algorithm and differentiated coordinate conversion strategies, ChartRecover achieves accurate and robust recovery of underlying

data across diverse chart types.

Our experimental results demonstrate that ChartRecover consistently maintains high stability and resilience under complex layouts and interference scenarios. The structured data extracted by the framework can directly support the mining of structure-property relationships and the evaluation of scientific chart credibility. Beyond improving the efficiency and reliability of scientific chart data acquisition, ChartRecover provides a practical pathway to strengthen research transparency, enable integrity verification, and facilitate large-scale reuse of scientific knowledge embedded in figures.

In the future, we will further enhance the precision of tick-mark localization in extremely small-target and low-resolution scenarios, and develop error modeling strategies for low-value data ranges and narrow-range data to improve numberical reconstruction accuracy. We also plan to extend ChartRecover to a broader range of scientific chart types beyond those currently supported, thereby robustness and general applicability, thereby further strengthening the robustness and applicability of ChartRecover. Overall, our findings demonstrate how automated chart data recovery can unlock previously inaccessible research value, enabling more rigorous validation and facilitating the broader reuse of scientific data and results.

Data availability

The datasets trained and tested during the current study are available in the Figshare repository, https://doi.org/10.6084/m9.figshare.32070000.

Code availability

The code supporting this study is available at https://github.com/materialsCnicCas/ChartRecover.

References

1. Huang, J., Chen, H., Yu, F. & Lu, W. From detection to application: Recent advances in understanding scientific tables and figures. ACM Comput. Surv. **56**, 1–39 (2024). URL https://doi.org/10.1145/3657285. DOI 10.1145/3657285.

2. Han, Y. et al. Automatic pipeline for information of curve graphs in papers based on deep learning. Int. J. Mach. Learn. Cybern. (2025). URL https://doi.org/10.1007/s13042-024-02496-7. DOI 10.1007/s13042-024-02496-7.

3. Yang, W., He, J., Zhang, X. & Gong, H. Ai-chartparser: A method for extracting experimental data from curve charts in academic papers. Comput. Graph. Forum e70146 (2025). URL https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.70146. DOI https://doi.org/10.1111/cgf.70146.

4. Huang, K.-H. et al. From pixels to insights: A survey on automatic chart understanding in the era of large foundation models. IEEE Educ. Activities Dep. **37**, 2550–2568 (2024). URL https://doi.org/10.1109/TKDE.2024.3513320. DOI 10.1109/TKDE.2024.3513320.

5. Mishchenko, A. & Vassilieva, N. Chart image understanding and numerical data extraction. In 2011 Sixth International Conference on Digital Information Management, 115–120 (2011). DOI 10.1109/ICDIM.2011.6093320.

6. Savva, M. et al. Revision: automated classification, analysis and redesign of chart images. In Proceedings of the 24th Annual ACM Symposium on User Interface Software and Technology, UIST '11, 393–402 (Association for Computing Machinery, New York, NY, USA, 2011). URL https://doi.org/10.1145/2047196.2047247. DOI 10.1145/2047196.2047247.

7. Al-Zaidy, R. A. & Giles, C. L. Automatic extraction of data from bar charts. In Proceedings of the 8th International Conference on Knowledge Capture, K-CAP '15 (Association for Computing Machinery, New York, NY, USA, 2015). URL https://doi.org/10.1145/2815833.2816956. DOI 10.1145/2815833.2816956.

8. Albuquerque, G., Lowe, T. & Magnor, M. Synthetic generation of high-dimensional datasets. IEEE Transactions on Vis. Comput. Graph. **17**, 2317–2324 (2011). DOI 10.1109/TVCG.2011.237.

9. Dai, W., Wang, M., Niu, Z. & Zhang, J. Chart decoder: Generating textual and numeric information from chart images automatically. J. Vis. Lang. Comput. **48**, 101–109 (2018). URL https://www.sciencedirect.com/science/article/pii/S1045926X18301162. DOI https://doi.org/10.1016/j.jvlc.2018.08.005.

10. Szegedy, C. et al. Going deeper with convolutions . In 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 1–9 (IEEE Computer Society, Los Alamitos, CA, USA, 2015). URL https://doi.ieeecomputersociety.org/10.1109/CVPR.2015.7298594. DOI 10.1109/CVPR.2015.7298594.

11. Cortes, C. & Vapnik, V. Support-vector networks. Mach. Learn. **20**, 273–297 (1995). URL https://doi.org/10.1023/A:1022627411411. DOI 10.1023/A:1022627411411.

12. Liu, X., Klabjan, D. & NBless, P. Data extraction from charts via single deep neural network. arXiv preprint arXiv:1906.11906 (2019).

13. Simonyan, K. & Zisserman, A. Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556 (2014).

14. Santoro, A. et al. A simple neural network module for relational reasoning. NIPS'17, 4974–4983 (Curran Associates Inc., Red Hook, NY, USA, 2017).

15. Shi, B., Bai, X. & Yao, C. An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition . IEEE Transactions on Pattern Analysis & Mach. Intell. **39**, 2298–2304 (2017). URL https://doi.ieeecomputersociety.org/10.1109/TPAMI.2016.2646371. DOI 10.1109/TPAMI.2016.2646371.

16. Ren, S., He, K., Girshick, R. & Sun, J. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks . IEEE Transactions on Pattern Analysis & Mach. Intell. **39**, 1137–1149 (2017). URL https://doi.ieeecomputersociety.org/10.1109/TPAMI.2016.2577031. DOI 10.1109/TPAMI.2016.2577031.

17. Carderas, A. et al. Automated data extraction of bar chart raster images. arXiv preprint arXiv:2011.04137 (2020).

18. Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J. & Wojna, Z. Rethinking the Inception Architecture for Computer Vision . In 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2818–2826 (IEEE Computer Society, Los Alamitos, CA, USA, 2016). URL https://doi.ieeecomputersociety.org/10.1109/CVPR.2016.308. DOI 10.1109/CVPR.2016.308.

19. Dong, C., Loy, C. C., He, K. & Tang, X. Image super-resolution using deep convolutional networks. IEEE Transactions on Pattern Analysis Mach. Intell. **38**, 295–307 (2016). DOI 10.1109/TPAMI.2015.2439281.

20. Poco, J. & Heer, J. Reverse-engineering visualizations: Recovering visual encodings from chart images. Comput. Graph. Forum **36**, 353–363 (2017). URL https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.13193. DOI https://doi.org/10.1111/cgf.13193. https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.13193.

21. Redmon, J., Divvala, S., Girshick, R. & Farhadi, A. You Only Look Once: Unified, Real-Time Object Detection . In 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779–788 (IEEE Computer Society, Los

Alamitos, CA, USA, 2016). URL https://doi.ieeecomputersociety.org/10.1109/CVPR.2016.91. DOI 10.1109/CVPR.2016.91.

22. Smith, R. An overview of the tesseract ocr engine. In Ninth International Conference on Document Analysis and Recognition (ICDAR 20 vol. 2, 629–633 (2007). DOI 10.1109/ICDAR.2007.4376991.

23. Shtok, J. et al. Charter: heatmap-based multi-type chart data extraction. arXiv preprint arXiv:2111.14103 (2021).

24. Duan, K. et al. CenterNet: Keypoint Triplets for Object Detection . In 2019 IEEE/CVF International Conference on Computer Vision (ICCV), 6568–6577 (IEEE Computer Society, Los Alamitos, CA, USA, 2019). URL https://doi.ieeecomputersociety.org/10.1109/ICCV.2019.00667. DOI 10.1109/ICCV.2019.00667.

25. Ma, W. et al. Towards an efficient framework for data extraction from chart images. In Lladós, J., Lopresti, D. & Uchida, S. (eds.) Document Analysis and Recognition – ICDAR 2021, 583–597 (Springer International Publishing, Cham, 2021). DOI 10.1007/978-3-030-86549-$8_3$7.

26. Cai, Z. & Vasconcelos, N. Cascade R-CNN: Delving Into High Quality Object Detection . In 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 6154–6162 (IEEE Computer Society, Los Alamitos, CA, USA, 2018). URL https://doi.ieeecomputersociety.org/10.1109/CVPR.2018.00644. DOI 10.1109/CVPR.2018.00644.

27. Long, J., Shelhamer, E. & Darrell, T. Fully convolutional networks for semantic segmentation . In 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 3431–3440 (IEEE Computer Society, Los Alamitos, CA, USA, 2015). URL https://doi.ieeecomputersociety.org/10.1109/CVPR.2015.7298965. DOI 10.1109/CVPR.2015.7298965.

28. Cliche, M., Rosenberg, D., Madeka, D. & Yee, C. Scatteract: Automated extraction of data from scatter plots. In Ceci, M., Hollmén, J., Todorovski, L., Vens, C. & Džeroski, S. (eds.) Machine Learning and Knowledge Discovery in Databases, 135–150 (Springer International Publishing, Cham, 2017). DOI 10.1007/978-3-319-71249-9_9.

29. Luo, J., Li, Z., Wang, J. & Lin, C.-Y. ChartOCR: Data Extraction from Charts Images via a Deep Hybrid Framework . In 2021 IEEE Winter Conference on Applications of Computer Vision (WACV), 1916–1924 (IEEE Computer Society, Los Alamitos, CA, USA, 2021). URL https://doi.ieeecomputersociety.org/10.1109/WACV48630.2021.00196. DOI 10.1109/WACV48630.2021.00196.

30. Lal, J., Mitkari, A., Bhosale, M. & Doermann, D. Lineformer: Line chart data extraction using instance segmentation. In Fink, G. A., Jain, R., Kise, K. & Zanibbi, R. (eds.) Document Analysis and Recognition - ICDAR 2023, 387–400 (Springer Nature Switzerland, Cham, 2023). DOI 10.1007/978-3-031-41734-$4_2$4.

31. P, S. V., Yusuf Hassan, M. & Singh, M. Lineex: Data extraction from scientific line charts. In 2023 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 6202–6210 (2023). DOI 10.1109/WACV56688.2023.00615.

32. Kanroo, M. S., Kawoosa, H. S., Rana, K. & Goyal, P. C3e: A framework for chart classification and content extraction. Comput. Electr. Eng. **121**, 109861 (2025). URL https://www.sciencedirect.com/science/article/pii/S0045790624007882. DOI https://doi.org/10.1016/j.compeleceng.2024.109861.

33. Yan, P., Ahmed, S. & Doermann, D. Context-aware chart element detection. In Fink, G. A., Jain, R., Kise, K. & Zanibbi, R. (eds.) Document Analysis and Recognition - ICDAR 2023, 218–233 (Springer Nature Switzerland, Cham, 2023). DOI 10.1007/978-3-031-41676-$7_1$3.

34. Numajiri, H. & Hayashi, T. Analysis on open data as a foundation for data-driven research. Sci. **129**, 6315–6332 (2024). URL https://doi.org/10.1007/s11192-024-04956-x. DOI 10.1007/s11192-024-04956-x.

35. LI, G. Ai4r: The fifth scientific research paradigm. Bull. Chin. Acad. Sci. **39**, 1–9 (2024). DOI 10.16418/j.issn.1000-3045.20231007002.

36. Whang, S. E. & Lee, J.-G. Data collection and quality challenges for deep learning. Proc. VLDB Endow. **13**, 3429–3432 (2020). URL https://doi.org/10.14778/3415478.3415562. DOI 10.14778/3415478.3415562.

37. Zhang, H. et al. DINO: DETR with improved denoising anchor boxes for end-to-end object detection. In The Eleventh International Conference on Learning Representations, 8587–8605 (2023). URL https://openreview.net/forum?id=3mRwyG5one.

38. Zhang, S. et al. Dense Distinct Query for End-to-End Object Detection . In 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 7329–7338 (IEEE Computer Society, Los Alamitos, CA, USA, 2023). URL https://doi.ieeecomputersociety.org/10.1109/CVPR52729.2023.00708. DOI 10.1109/CVPR52729.2023.00708.

39. Zong, Z., Song, G. & Liu, Y. DETRs with Collaborative Hybrid Assignments Training . In 2023 IEEE/CVF International Conference on Computer Vision (ICCV), 6725–6735 (IEEE Computer Society, Los Alamitos, CA, USA, 2023). URL https://doi.ieeecomputersociety.org/10.1109/ICCV51070.2023.00621. DOI 10.1109/ICCV51070.2023.00621.

40. Chen, K. et al. Mmdetection: Open mmlab detection toolbox and benchmark. arXiv preprint arXiv:1906.07155 (2019).

41. Davila, K. et al. Icpr 2022: Challenge on harvesting raw tables from infographics (chart-infographics). In 2022 26th International Conference on Pattern Recognition (ICPR), 4995–5001 (2022). DOI 10.1109/ICPR56361.2022.9956289.

42. Davila, K. et al. Icpr 2020 - competition on harvesting raw tables from infographics. In Del Bimbo, A. et al. (eds.) Pattern Recognition. ICPR International Workshops and Challenges, 361–380 (Springer International Publishing, Cham, 2021). DOI 10.1007/978-3-030-68793-9$_2$7.

43. Davila, K. et al. Chart-info 2024: A dataset for chart analysis and recognition. In Antonacopoulos, A. et al. (eds.) Pattern Recognition, 297–315 (Springer Nature Switzerland, Cham, 2025).

44. Rigaud, C., Doucet, A., Coustaty, M. & Moreux, J.-P. Icdar 2019 competition on post-ocr text correction. In 2019 International Conference on Document Analysis and Recognition (ICDAR), 1588–1593 (2019). DOI 10.1109/ICDAR.2019.00255.

45. Vaswani, A. et al. Attention is all you need. NIPS'17, 6000–6010 (Curran Associates Inc., Red Hook, NY, USA, 2017). DOI 10.5555/3295222.3295349.

46. Carion, N. et al. End-to-end object detection with transformers. In Vedaldi, A., Bischof, H., Brox, T. & Frahm, J.-M. (eds.) Computer Vision – ECCV 2020, 213–229 (Springer International Publishing, Cham, 2020). DOI 10.1007/978-3-030-58452-8$_1$3.

47. Liu, Z. et al. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows . In 2021 IEEE/CVF International Conference on Computer Vision (ICCV), 9992–10002 (IEEE Computer Society, Los Alamitos, CA, USA, 2021). URL https://doi.ieeecomputersociety.org/10.1109/ICCV48922.2021.00986. DOI 10.1109/ICCV48922.2021.00986.

48. Zhang, S., Chi, C., Yao, Y., Lei, Z. & Li, S. Z. Bridging the Gap Between Anchor-Based and Anchor-Free Detection via Adaptive Training Sample Selection . In 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR),

9756–9765 (IEEE Computer Society, Los Alamitos, CA, USA, 2020). URL https://doi.ieeecomputersociety.org/10.1109/CVPR42600.2020.00978. DOI 10.1109/CVPR42600.2020.00978.

49. Siegel, N., Horvitz, Z., Levin, R., Divvala, S. & Farhadi, A. Figureseer: Parsing result-figures in research papers. In Leibe, B., Matas, J., Sebe, N. & Welling, M. (eds.) Computer Vision – ECCV 2016, 664–680 (Springer International Publishing, Cham, 2016). DOI 10.1007/978-3-319-46478-7$_4$1.

50. Kato, H., Nakazawa, M., Yang, H.-K., Chen, M. & Stenger, B. Parsing Line Chart Images Using Linear Programming . In 2022 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2553–2562 (IEEE Computer Society, Los Alamitos, CA, USA, 2022). URL https://doi.ieeecomputersociety.org/10.1109/WACV51458.2022.00261. DOI 10.1109/WACV51458.2022.00261.

Funding Statement

This work was supported by National Key R&D Program of China(No.2025YFE0102600), the Research Projects of Ganjiang Innovation Academy, Chinese Academy of Sciences (No.E455F001), and the Strategic Priority Research Program of the Chinese Academy of Sciences(No.XDA0410404).

Author information

Authors and Affiliations

Computer Network Information Center, Chinese Academy of Sciences, Beijing, 100083, China

Yang Yuan, Sihan Liang, Ji Zhang, Yangang Wang & Zongguo Wang

Contributions

Y.Y. and S.L. conceived the research. Y.Y. and J.Z. performed the experiments and analyzed the data. Y.Y. and S.L. wrote the manuscript. Y.W. and Z.W. revised the manuscript. All authors read and approved the final paper.

Corresponding author

Correspondence to Zongguo Wang.

Ethics declarations

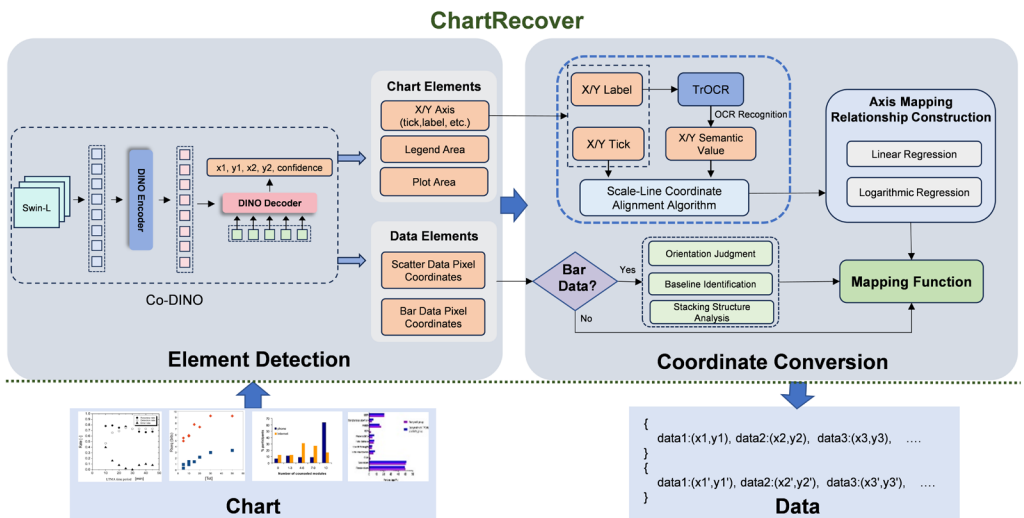
Competing interests

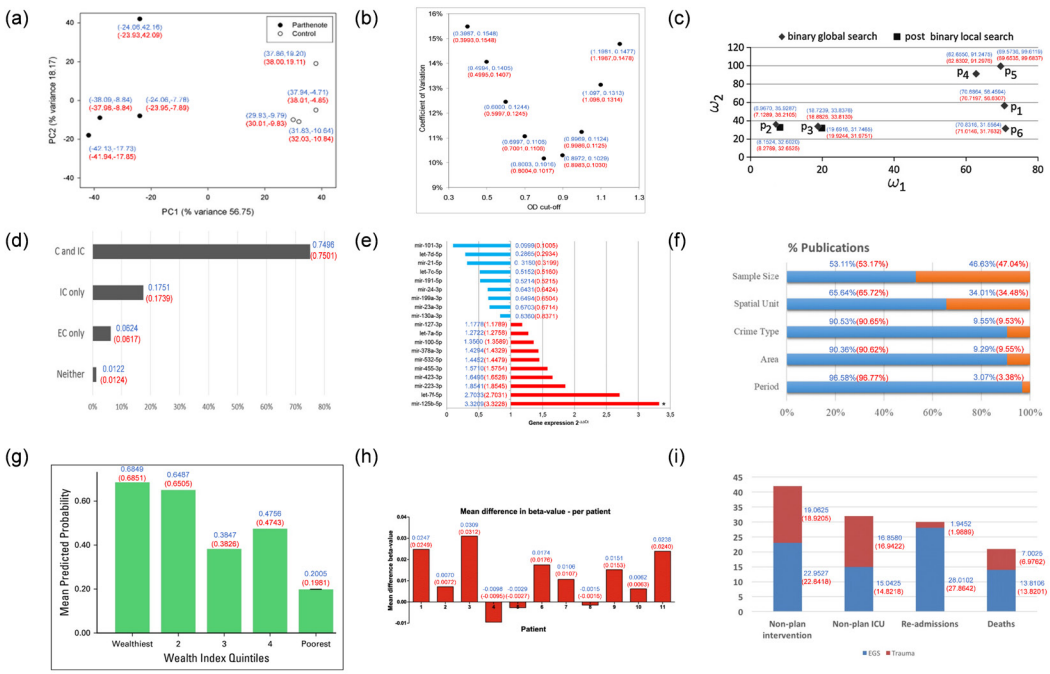
The authors declare no competing interests.

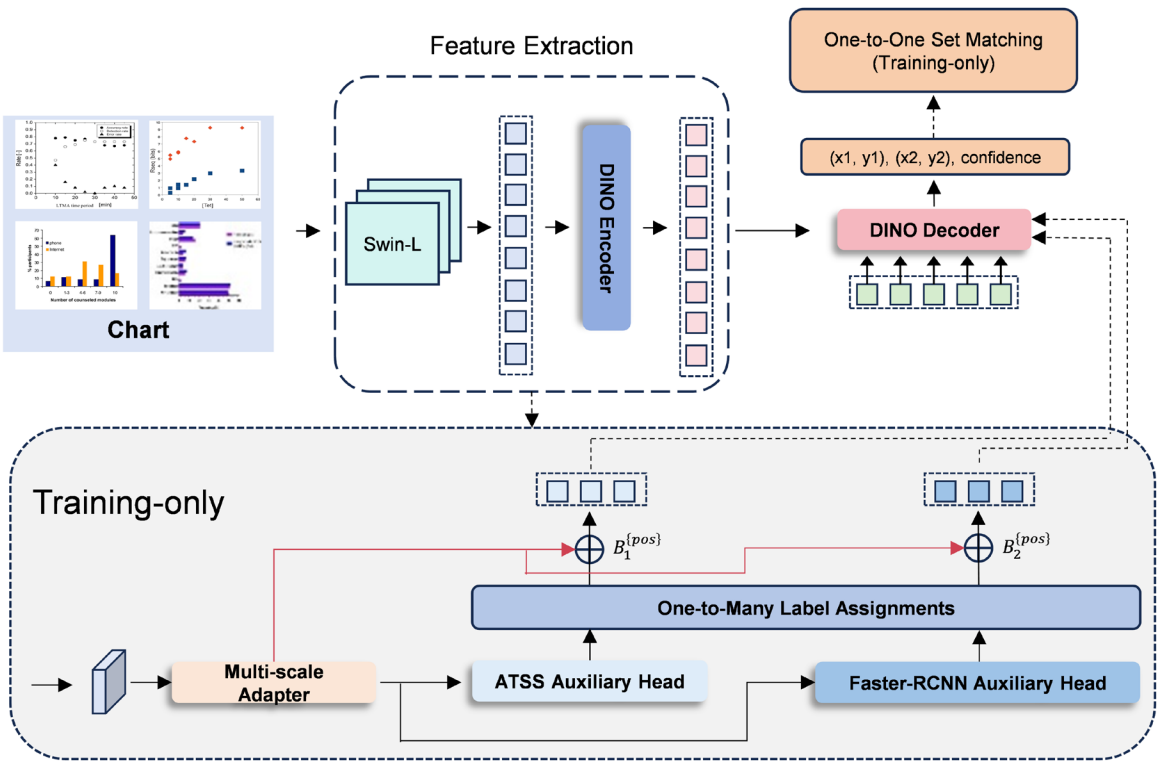
Figure 2. Evaluation of data point coordinate extraction results by the ChartRecover framework across various chart types: (a-c) scatter charts; (d, e) horizontal bar charts; (f) stacked horizontal bar chart; (g, h) vertical bar charts; and (i) stacked vertical bar chart. The blue points represent the real coordinate values, while the red points represent the coordinate values predicted by the framework. (All images are from the PMC dataset[41])

Figure 3. Object detection model architecture diagram. Following the Co-DINO framework, auxiliary heads utilize ATSS and Faster-RCNN. The dashed arrows represent the training process.

Figure 4. Diagram of various types of bar charts: (a) conventional horizontal bar chart; (b) horizontal bar chart with left-right baseline distribution; (c) stacked horizontal bar chart; (d) conventional vertical bar chart; (e) vertical bar chart with top-bottom baseline distribution; and (f) stacked vertical bar chart. All chart examples are sourced from the PMC dataset[41].






Feature Extraction
One-to-One Set Matching
(Training-only)
(x1, y1), (x2, y2), confidence
DINO Decoder
Swin-L
DINO Encoder
Chart
Training-only
$B_1^{\{pos\}}$
$B_2^{\{pos\}}$
One-to-Many Label Assignments
Multi-scale
Adapter
ATSS Auxiliary Head
Faster-RCNN Auxiliary Head